

Aula 38 - Princípios SOLID

Descrição: Nesta aula vamos entender profundamente os princípios SOLID focando principalmente no SRP e OCP.

O **SOLID** é um acrônimo criado por **Michael Feathers** para condensar cinco princípios de design de software orientado a objetos que foram apresentados por **Robert C. Martin** (conhecido como "Uncle Bob"). Eles servem como um guia para criar códigos fáceis de manter, estender, testar e adaptar ao longo do tempo, evitando que o software se transforme em um "sistema legado monolítico e frágil". Aqui está a explicação detalhada de cada um dos cinco princípios:

S — Single Responsibility Principle (Princípio da Responsabilidade Única)

"Uma classe deve ter um, e apenas um, motivo para mudar."

Uma classe não deve ser uma "ferramenta multifuncional". Ela deve ser especialista em uma única área de negócio ou papel dentro do sistema. Quando uma classe faz coisas demais (acumula responsabilidades), qualquer alteração em uma regra de negócio pode quebrar, acidentalmente, outra funcionalidade não relacionada.

- **Violação comum:** Uma classe `Usuario` que contém as propriedades do usuário, o método para salvar o usuário no banco de dados e o método para enviar um e-mail de boas-vindas.
 - **Solução SOLID:** Separar em três classes: `Usuario` (dados), `UsuarioRepository` (banco de dados) e `EmailService` (comunicação).
-

O — Open/Closed Principle (Princípio do Aberto/Fechado)

"Entidades de software devem estar abertas para extensão, mas fechadas para modificação."

Você deve ser capaz de adicionar novos comportamentos ou funcionalidades ao sistema sem precisar alterar o código-fonte que já está escrito, testado e funcionando. Isso previne a introdução de novos bugs em recursos antigos. Esse princípio é atingido principalmente através do uso de **interfaces** e **polimorfismo**.

- **Violação comum:** Uma classe `CalculadoraDeImposto` com um método cheio de `if/else` ou `switch` testando o tipo de produto para aplicar a alíquota correspondente. Toda vez que um novo produto surgir, você terá que modificar essa classe.
 - **Solução SOLID:** Criar uma interface `ImpostoInterface` com o método `calcular()`. Cada novo produto será uma nova classe que implementa essa interface. A `CalculadoraDeImposto` passa a depender apenas da interface, ficando blindada contra modificações futuras.
-

L — Liskov Substitution Principle (Princípio da Substituição de Liskov)

"As classes derivadas devem ser substituíveis por suas classes base."

Formulado por Barbara Liskov, este princípio diz que se uma classe **B** é herdeira (subclasse) de uma classe **A**, então nós deveríamos ser capazes de substituir **A** por **B** em qualquer parte do código sem que o sistema quebre ou mude de comportamento inesperadamente. A herança deve expressar uma relação real de comportamento, e não apenas de semelhança semântica.

- **Violação comum:** O clássico exemplo do Retângulo e do Quadrado. Se você criar uma classe **Quadrado** que herda de **Retângulo**, ao tentar alterar apenas a largura do "Quadrado", ele forçará a alteração da altura também (para manter os lados iguais). Se o seu código esperava o comportamento de um retângulo padrão, ele vai quebrar. Outra violação clássica é criar uma subclasse que lança uma exceção do tipo **MétodoNãoImplementadoException** para um método da classe pai.
 - **Solução SOLID:** Se a subclasse não consegue fazer tudo o que a classe pai faz de forma natural, a herança está errada. Use composição ou interfaces mais específicas em vez de herança forçada.
-

I — Interface Segregation Principle (Princípio da Segregação de Interface)

"Uma classe não deve ser forçada a depender de interfaces que ela não utiliza."

É melhor criar várias interfaces específicas e pequenas do que uma única interface genérica e "gorda". Quando uma interface possui métodos demais, as classes que a implementam são frequentemente obrigadas a criar métodos vazios apenas para satisfazer o contrato do compilador.

- **Violação comum:** Uma interface **AveInterface** que possui os métodos **bicar()**, **chocarOvos()** e **voar()**. Se você criar uma classe **Pinguim** que implementa essa interface, será obrigado a implementar o método **voar()**, mesmo que pinguins não voem.
 - **Solução SOLID:** Dividir em interfaces menores, como **AveInterface** (com os métodos comuns) e **AveVoadoraInterface** (apenas com o método **voar()**). O pinguim implementa apenas a primeira.
-

D — Dependency Inversion Principle (Princípio da Inversão de Dependência)

"Dependa de abstrações, não de implementações concretas."

Módulos de alto nível (as regras de negócio principais) não devem depender de módulos de baixo nível (detalhes de infraestrutura, como bancos de dados, APIs externas ou bibliotecas). Ambos devem depender de abstrações (interfaces). Isso desacopla o sistema e torna as peças facilmente substituíveis.

- **Violação comum:** Dentro da sua classe de negócio principal **ProcessadorDePedido**, você instancia diretamente a classe do banco de dados: `$banco = new MySQLConnection();`. Seu código principal ficou totalmente amarrado e dependente do MySQL.

- **Solução SOLID:** O `ProcessadorDePedido` deve exigir no seu construtor uma interface chamada `DbConnectionInterface`. Quem decide se vai passar uma conexão MySQL, PostgreSQL ou MongoDB é quem está chamando a classe de fora (Injeção de Dependência).

Resumo Visual do SOLID

	Princípio	Em poucas palavras...
S	Responsabilidade Única (SRP)	Cada classe cuida apenas do seu próprio quadrado.
O	Aberto/Fechado (OCP)	Adicione novas funções criando novo código, não alterando o antigo.
L	Substituição de Liskov (LSP)	Uma classe filha deve honrar e conseguir fazer tudo o que a classe pai fazia.
I	Segregação de Interface (ISP)	Não force uma classe a implementar algo que ela não precisa.
D	Inversão de Dependência (DIP)	Amarre seu código a contratos (interfaces), não a tecnologias específicas.

Fechamento:

Para fechar com chave de ouro e consolidar nosso aprendizado sobre o **SOLID**, pense nesses cinco princípios não como regras rígidas de preenchimento de tabela, mas como um **investimento na sua paz de espírito profissional**.

Dominar o SRP, OCP e os demais pilares transforma a sua relação com o código de duas formas fundamentais:

- **Do ponto de vista técnico:** Você deixa de escrever códigos em bloco (onde mexer em uma linha parece desarmar uma bomba) e passa a construir sistemas baseados em "blocos de Lego", onde encaixar uma nova peça (como uma nova API de pagamento ou um novo relatório) é natural, seguro e rápido.
- **Do ponto de vista de carreira:** Você para de apenas "codificar" e passa a **arquitetar** soluções. Um desenvolvedor que domina o SOLID gasta menos tempo caçando bugs inexplicáveis e mais tempo criando softwares escaláveis que sobrevivem ao crescimento das empresas.

A melhor maneira de fixar esses conceitos é o treino ativo: da próxima vez que abrir um arquivo legível e sentir aquela "dor de cabeça" ao ver um método com dezenas de linhas e condicionais, tente isolar o problema.

Pergunte-se: *"Como posso quebrar isso em uma nova classe (SRP)?"* ou *"Como uma interface me ajudaria a eliminar esse excesso de if/else (OCP)?"*. É esse exercício diário que transforma teoria em excelência.

Que o seu próximo código seja tão limpo que sirva de documentação!